- **从 PPT 中摘抄的部分知识点，仅供同学复习梳理知识点参考；**
- **有些概念不全，或用词不精确，请同学们通过认真复习，进行修改、补全，做到认真复习，全面掌握知识点课程目标。**

1. 汇编语言编程时采用十六进制数，是由于其与计算机中数据存储、处理的单位长度适用；
2. 对一个二进制数按位求反后在末位加 1 的运算是求补运算；
3. BCD 码是一种用二进制编码的十进制数；
4. 80386 以上 CPU 具有 3 种运行模式：实模式、保护模式和虚拟 8086 模式；
5. 在实模式下，内存寻址方式由 16 位段寄存器和 16 位偏移地址形成 20 位的物理地址；
6. JZ xxx 指令的本质是当标志位 ZF=1 时转移到目标指令；
7. 偏移地址是指段内相对于段起始地址的 16 位位移量；
8. 机器指令给出的地址是逻辑地址；
9. 汇编程序设计面向计算机硬件；
10. 汇编语言的真实指令和机器指令面向计算机硬件；
11. 汇编语言的真实指令和机器指令是一对一的关系；
12. 汇编语言的每条真实指令是对寄存器、存储单元的具体操作；
13. 计算机所能执行的所有指令的集合是该计算机的指令系统；
14. 寻址方式是取得操作数地址的方式；
15. 计算机中一般寻址方式有：与操作数据有关的寻址方式、与转移地址有关的寻址方式；
16. 操作数通常保存在：指令中、CPU 的寄存器中、内存单元中、I/O 接口寄存器中；
17. 对 IO 设备的控制或数据传输实质上是对 I/O 接口寄存器的数据读写；
18. 80X86 的机器指令格式一般为：操作码　目的操作数　源操作数；
19. 有效地址 EA（偏移地址）的四种成分是：基址、变址、比例因子、位移量；
20. 有效地址 EA（偏移地址）的计算公式为：EA=基址+（变址*比例因子）+位移量；
21. 指令中操作数的有效地址 EA（偏移地址）表示形式：[基址][变址*比例因子][位移量] 或者 [基址+变址*比例因子+位移量]；
22. 指令中操作数的地址表示形式：段寄存器名：[基址+变址*比例因子+位移量] 或者段寄存器名：[基址][变址*比例因子][位移量]；

23. 转移指令和调用指令的目的地址位置有：段内转移和段间转移两种；
24. 转移指令和调用指令的目的地址位置确定操作本质是重新设置代码段寄存器 CS 和指令指针寄存器 IP 的内容；
25. 转移指令和调用指令的寻址方式有：段内寻址、段间寻址、直接寻址、段间寻址；
26. 转移指令和调用指令的段内寻址：转移指令与转向的目标指令在同一代码段中，CS 内容不变，IP 内容修改；
27. 转移指令和调用指令的段间寻址：转移指令与转向的目标指令在两个代码段中，CS 和 IP 内容修改；
28. 转移指令和调用指令的直接寻址：转向的目标指令地址由转移指令直接指明；
29. 转移指令和调用指令的段间寻址：转向的目标指令地址由转移指令中的寄存器或存储单元内容给出；
30. 位移量是相对某一点的距离，带符号数；
31. 偏移量（有效地址）是相对段基地址的正向位移量，无符号数；
32. 段间间接寻址是用存储器中的两个相继字内容来取代 IP 和 CS 寄存器中的当前内容；
33. 条件转移指令只能使用段内直接寻址方式；
34. 无条件转移（JMP）和无条件子程序调用（CALL）可用四种方式的任何一种；
35. MOV 指令中不能直接是两个存储单元之间数据传送，不影响标志位；
36. 进栈指令 PUSH SRC，其具体执行操作是 SP ← SP - 2、(SS*16+SP+1, SS*16+SP) ← SRC；
37. 出栈指令 POP DST，其具体执行操作是 DST ← (SS*16+SP+1, SS*16+SP)， SP ← SP + 2；
38. 堆栈：“后进先出”的存储区，存在于堆栈段中，SS*10H+SP 在任何时候都指向栈顶；
39. PUSH、POP 指令一般配对使用；
40. I/O 输入指令中只能使累加器寄存器和外设端口寄存器的之间的数据传输；
41. IO 设备的控制和数据传输实质上是对 IO 接口中 IO 寄存器内容的读写；
42. 乘法指令中字操作，即两个字相乘，其乘积存放在 DX:AX 中；
43. 十进制调整指令一般跟在对十进制 BCD 数的一般运算指令之后；
44. MOVS 单个串处理指令执行时将操作数地址的变址指针根据修改方向自动修改；且寄存器只能使用累加器；

45. 串处理指令前应先建立地址指针、方向标志；
46. 条件转移指令只能段内转移；
47. X86CPU 中 JCXZ 指令是 CX 寄存器内容为零则转移；
48. 循环指令 LOOP OPR 中的循环入口地址寻址方式中只能是短转移，不能近转移或跨越段；
49. 循环指令 LOOP OPR 不影响标志位；
50. CALL 指令与转移指令的转移执行方式相同，不同之处是 CPU 会自动将返回点(IP,CS)保存在堆栈中；
51. 带立即数的子程序返回指令中，立即数主要是为了修正堆栈的栈顶指针 SP 内容；
52. 中断向量是中断处理子程序的入口地址；
53. 汇编语言指令包括三类：机器指令、伪指令、宏指令；
54. 每条机器指令语句都生成机器代码，各对应一种 CPU 操作，在程序运行时由计算机执行；
55. 伪操作由汇编程序（汇编工具）解释执行，机器指令由 CPU 执行.EXE 程序时执行；
56. 伪指令（又称为伪操作）相当于声明指令。伪指令在汇编程序对源程序汇编期间仅由汇编程序按功能说明处理，以完成处理器选择、定义程序模式、定义数据、分配存储区、指示程序开始/结束等；
57. 伪指令（又称为伪操作）是告诉汇编程序的某些功能说明或定义，仅在汇编时解释执行，不会汇编成任何机器指令；
58. 宏指令包括：用户自定义宏指令和标准宏指令，一条指令对应一组机器指令，汇编时展开；
59. 汇编程序设计中机器指令语句和伪操作语句的区别和作用是什么？
60. 宏和子程序有什么不同之处？宏有什么用途？
61. 宏定义体中标号为什么在展开时要重新给新标号名，如何给出新标号？
62. 用汇编语言编写程序可以使用两种基本格式。完整段定义、简化段定义；
63. 所有的变量和标号都有三种属性：段基值(SEG)，偏移量(OFFSET)，类型(TYPE)：变量（字节/字/双字/四字/十字节）、标号（NEAR / FAR）；
64. 指令 mov ax, offset word_array 的寻址方式是：立即数寻址方式，执行结果是将变量 word_array 的偏移地址送 AX 累加器寄存器；

65. 指令 mov ax, word_array 的寻址方式是：直接寻址方式，执行结果是将变量 word_array 的值送 AX 累加器寄存器；
66. 变量的属性在程序中可以通过两种方式改变：LABEL 伪操作重新定义、指令中 PTR 指定的变量类型属性；
67. 表达式赋值伪操作 EQU 给符号名不分配占用存储单元，只是相当定义了一个常数的数值；
68. 在汇编程序对源程序汇编过程中，使用地址计数器保存当前正在汇编的基本处理单位在存储器中的首地址（如指令的首地址，操作数首地址）；
69. 指令中地址计数器的值用‘$’来表示，汇编语言允许用户直接用‘$’来引用地址计数器的值；
70. ORG 伪操作用来设置当前地址计数器的值，即分配后续数据、指令的存储器开始地址；
71. 标号经常在转移或 CALL 指令的操作数字段出现，用以表示转向地址；
72. 汇编语言指令中操作数项可以是常数（立即数）/表达、寄存器、存储器地址/表达、标号/变量式；
73. 汇编语言的上机过程包括哪几步？并解释每一步的作用。
74. 用 debug 调试程序时存储单元显示数据如下，则物理地址 076B0H 存储的字节数据是 0CH，076BEH 存储的字数据是 002CH。

```
-d 076A:0020
076A:0020  21 00 20 00 37 00 1F 00-5F
```

```
-d 076A:0020
076A:0020  21 00 20 00 37 00 1F 00-5F 00 45 00 5B 00 2C 00
```

```
-d 076A:0000
076A:0000  0C 00 20 00 02 00 3B 00-DE 00 5B 00 17 00 2C 00
```

75. 循环程序中被循环多次执行的指令段称为循环体；
76. 循环程序设计适应于处理算法相同，每次处理时需要有规律地改变数据或数据地址的问题；
77. 循环程序包括三个基本组成部分：初始设置、循环体、循环控制转移；
78. 循环程序初始设置考虑因素：循环次数，数据和变址指针初始化等；
79. 循环程序的循环体考虑因素：根据任务，选择算法；修改指针等；设置循环控制转移其他条件；

80. 循环程序的循环控制转移考虑因素：正确选择条件转移指令，基于循环次数、其他条件，如 FLAGS；

81. 汇编程序返回 DOS 操作系统有两种实现方法；

82. 汇编程序结束时两条指令 MOV AX, 4C00H 和 INT 21H 的作用是正常返回 DOS 操作系统；

83. 简述根据逻辑尺方法实现多分支转移的方法。适合对有规律的数组进行无规律操作等；

84. 多重循环程序结构是循环中有循环，内外层循环都应该具有完整的循环程序结构；

85. CT EQU ($-ARY)/2 只是定义了一个常数，不分配存储单元；

86. CT db ($-ARY)/2 为变量分配一个字节存储单元，并赋初值，变量名为 CT；

87. 下面定义后常数 CT 的值是 18 或 12H

```
ARY  dw 1,2,3,4,5,6,7,8,9
CT EQU $-ARY
```

(1) 若 DS=0001H，通过下面定义后，则存储单元(0A013H)中的数值是 87 H，常数 CT 的值是 18 或 12H

```
        ORG  0A000H
XYZ   DD 87654321H
ARY   DW 1, 2, 3, 4, 5, 6, 7, 8, 9
CT    EQU $-ARY
```

88. 利用跳跃表法实现多路分支，关键是：构建跳转表（分支转移目标地址表）、灵活、正确使用无条件间接转移指令实现跳转；

89. 若跳转表的地址数据定义：用 DW 定义 ←→ 段内间接转移指令（IP），用 DD 定义 ←→ 段间间接转移指令（IP, CS）；

90. 调用程序和子程序在同一代码段中，则使用 NEAR 属性；

91. 调用程序和子程序不在同一代码段中，则使用 FAR 属性；

92. 汇编程序设计时，调用程序和子程序在同一代码段中，则使用 NEAR 属性；

93. 汇编程序设计时，调用程序和子程序不在同一代码段中，则使用 FAR 属性。

94. 过程定义时属性 NEAR、FAR，汇编工具汇编时过程返回指令 RET 会被转换为对应的段内、段间返回指令 RET、RETF；

95. 80X86 汇编程序在汇编时用子程序属性确定 CALL 和 RET 指令属性，用户只需在定义过程时考虑属性， CALL 和 RET 指令属性可以不考虑让汇编程序确定。
96. FAR 属性子程序可以被同一段内或不同段内程序调用，而 NEAR 属性子程序只能被同一段内程序调用；
97. DOS 和 BIOS 功能调用的软中断 int n （n : 中断类型号）指令执行的操作有：自动保存现场（FLAGS）和返回地址（CS,IP）；清除中断标志位，关中段； 转中断处理程序；
98. 调用子程序时参数传送的一般途径是通过：寄存器、存储器，即参数放在主程序和子程序都可以访问到的地方；
99. 调用子程序时变量参数传送的具体方法有：通过寄存器传送变量、通过存储器传送变量、通过地址表传送变量地址、通过堆栈传送变量或变量地址；
100. 指令 mov table+2, offset count（count dw ）的结果是将立即数（变量 count 的有效地址）送到 table+2 给出的直接有效地址的两个存储器单元；
101. 调用子程序时参数传送方法确定原则是：参数少时传送参数，参数多时传送参数地址；
102. 子程序嵌套是一个子程序作为调用程序调用另一个子程序；
103. 递归调用是子程序调用的子程序是它自身；
104. BIOS 和 DOS 系统功能调用一般采用软中断方式；
105. 宏定义是将源程序中一段有独立功能的程序代码定义为宏，可以多次调用宏，调用时使用一条宏指令语句就即可；
106. 宏调用是使用用户自定义的宏指令，可将多次使用的功能用一条宏指令来代替；
107. 宏展开是在汇编时，用宏指令定义的程序段替代宏指令；
108. 对宏指令的调用，必须先定义后调用；
109. 宏指令调用的实元可以是常数、寄存器、存储单元名、地址、表达式；也可以是操作码或操作码的一部分；
110. 多次宏调用展开时，为了解决标号冲突问题，由 LOCAL 定义的标号也由 ??0000～??FFFF 偏移量替代 （其实质是自动给了一个新标号）；
111. 源程序被汇编时，汇编程序将对每个宏调用作宏展开；
112. 宏展开时用宏定义体替换宏指令名，把宏定义体复制到调用宏指令的位置上，同时用实元取代哑元；

113.子程序在程序执行期间被调用，只占用自身大小的一段空间；优点是模块化，节省内存，可被多次调用，编程效率高；

114.宏调用在汇编期间被展开,每调用一次就把宏定义体展开一次。优点是参数传送简单，执行效率高；不节省空间；适用于子功能代码较短、传递参数较多的情况。

115.CPU 与外设之间交换的信息包括：数据信息、控制信息、状态信息；

116.从程序设计的角度看，IO 接口由一组寄存器组成，是完成设备数据输入输出和控制的桥梁；

117.从程序设计的角度看，完成外部 IO 设备数据输入输出和控制，就是对 IO 接口寄存器的读写；

118.为了访问接口上的寄存器，系统给这些寄存器分配专门的存取访问地址，这样的地址称为 I/O 端口地址；

119.如果 I/O 端口地址和存储单元的地址是各自独立编址的，则存取接口寄存器中的数据要使用专门的 I/O 指令完成；

120.I/O 设备的数据传送方式一般有：程序直接控制 I/O 方式（无条件传送方式、查询方式）、直接存储器存取（DMA)方式、中断方式；

121.无条件传送方式默认外设总是处在“准备好” 或者“不忙” 状态；

122.查询方式在输入输出之前首先查询外设的状态是否处于“准备好”（输入方式下）或者“不忙”（输出状态下）；

123.I/O 设备在输入输出时，当外设状态处于“准备好”（输入方式下）或者“不忙”（输出状态下），才可以进行输入/输出；

124.计算机中当出现异常事件或事先安排好的事件，迫使 CPU 暂时中止现行程序的执行，转去执行另一处理程序，这个过程称为中断；

125.引起中断的事件称为中断源；

126.外设控制器或协处理器引起的中断称为硬件中断（或外中断）；

127.程序中的中断指令 INT 或 CPU 错误结果产生的中断称为软件中断或内中断；

128.硬件非屏蔽中断是不受中断允许标志位 IF 屏蔽的中断，中断类型号=2

129.硬件可屏蔽中断只有当标志寄存器的中断屏蔽标志 IF 为 1 时，才能引起中断；

130.标志寄存器的标志为 IF 作用是屏蔽或允许 CPU 响应可屏蔽中断；

131.软中断指令 INT N 的中断不受中断屏蔽标志 IF 影响，是不可屏蔽的中断；

132. 8259A 中与中断相关寄存器有：中断屏蔽寄存器(IMR) （屏蔽单个中断请求）、中断命令寄存器（设置中断优先级等）；
133. CPU 可以响应某设备的中断服务请求的条件是：中断屏蔽寄存器中对应位=0，同时 IF=1；
134. 屏蔽所有可屏蔽的外部设备中断请求：用标志寄存器中的中断屏蔽标志位 IF。开中断指令 STI 设置中断允许位（IF=1），关中断指令 CLI 清除中断允许位（IF=0）；
135. 屏蔽单个可屏蔽的外部设备中断请求：用 8259A 中的中断屏蔽寄存器对应屏蔽位，OUT 指令设置相应位为 1 屏蔽中断请求；
136. 禁止单个中断源的中断请求：用 OUT 指令设置设备接口中控制寄存器对应位。
137. 中断向量表是各类型中断处理程序的入口地址表；
138. 中断过程包括：中断响应、中断处理、中断返回；
139. 中断响应时由 CPU 会自动将标志寄存器(FLAGS)内容入栈、保存被中断程序断点、禁止硬件中断和单步中断、转中断服务处理程序入口地址；
140. 中断处理程序设计要考虑保存现场、恢复现场等；
141. 子程序调用时 CPU 自动只把 CS 和 IP 压入堆栈；中断响应时 CPU 自动把 CS 和 IP 压入堆栈外，还把标志寄存器的内容压入堆栈，并且关掉了中断和单步运行方式。**有些概念不全，注意自己要补全！**
142. 子程序返回时只把断点地址从堆栈弹出送 CS 和 IP；中断返回时除恢复断点地址 CS 和 IP 外，还要恢复标志寄存器的内容；
143. 中断一般随机发生，软中断在程序中预先安排；
144. 子程序调用在程序中预先安排调用；
145. 子程序调用只把 CS 和 IP 压入堆栈，而中断响应除把 CS 和 IP 压入堆栈外，还把标志寄存器的内容压入堆栈，并且关掉了中断和单步运行方式；
146. 子程序返回只把断点地址从堆栈弹出送 CS 和 IP，而中断返回除恢复断点地址 CS 和 IP 外，还要恢复标志寄存器的内容；
147. 中断一般随机发生，软中断在程序中预先安排，子程序调用在程序中预先安排调用；
148. 用户自定义中断时要在主程序中设置中断向量表中相应的中断向量，正确设置中断允许、屏蔽位和优先级
149. 中断优先级(Priority)：当多个中断源同时向 CPU 请求中断时，CPU 确定优先处理的次序；

150.80X86 中规定的中断优先级次序为：软件中断（除法错，INTO，INT）、非屏蔽中断（NMI）、可屏蔽中断（INTR）、单步中断；

151.BIOS 是固化在 ROM 中的基本输入输出系统 BIOS；

152.BIOS 和 DOS 功能调用采用软中断指令 INT n 中断方式，采用软件中断的方式实现功能调用；

153.DOS 功能调用采用软中断指令 INT 21 中断方式；

154.编写应用程序推荐调用 I/O 的顺序为：DOS--> BIOS--> 直接外设接口访问；

155.键盘硬件中断是如果有键按下，且中断也允许时，向 CPU 发中断，由 BIOS 中的键盘外设中断程序处理，转换成字符码，并将字符码和扫描码存储在内存缓冲区中；

156.键盘软件中断是应用程序可以使用 DOS 和 BIOS 软件中断调用获得存储在内存缓冲区中的字符码和扫描码；

157.键盘软件中断和键盘硬件中断在 BIOS 中由不同的中断处理程序完成，功能不同；

158.键盘硬件中断是：如果有键按下，且中断也允许，则向 CPU 发中断，由 BIOS 中的键盘外设中断程序处理，转换成字符码，并将字符码和扫描码存储在内存缓冲区中；

159.键盘软件中断是：应用程序可以使用 DOS 和 BIOS 软件中断调用获得存储在内存缓冲区中的字符码和扫描码；

160.键盘软件中断和键盘硬件中断在 BIOS 中由不同的中断处理程序完成，功能不同；

161.BIOS 的键盘硬件中断处理程序将扫描码转换为字符码，并将字符码和扫描码存储在内存的键盘缓冲区 KB_BUFFER 中；

162.BIOS 键盘软中断（INT 16H）是从内存的键盘缓冲区 KB_BUFFER 中读取键盘字符或状态字；

163.显示存储器作为系统存储器的一部分，可用访问普通内存的方法访问显示存储器；

164.每个显示字符占 2 个显示存储单元，分别存放要显示字符的字符码（如 ASCII 码）和字符显示属性码；

165.显示器 BIOS 中断调用(INT 10H)实质是对显示存储区的管理和操作；

166.计算机与常用外部设备（如打印机、键盘、显示器）的数据读写一般采用的字符码是 ASCII 码；